

The Frobenius Loop: A Permanently Paradoxical Fixed Point in the Paraconsistent Universal Engine

umpolungfish

2026-05-21

Abstract

We exhibit a three-instruction circular program on the TriPhaseRegister machine—the Universal Engine extended with Belnap FOUR belief tracking—that runs indefinitely with all registers permanently in the *Both* (B) state. The loop invariant is a computational realization of the Frobenius identity $\mu \circ \delta = \text{id}$: FFUSE applied to two B-state outputs of FSPLIT returns B, preventing collapse to any classical truth value. We prove that the paradox count after n complete cycles is exactly $P(n) = 4n$, and confirm this algebraic prediction empirically at 1.9×10^{10} steps of continuous execution.

1 The Machine

Definition 1 (TriPhaseRegister). A *TriPhaseRegister* is a pair (flux, val) where $\text{flux} \in \{00, 01, 10, 11\}$ and $\text{val} \in \{\perp, \text{FIXED}\}$. The flux encodes a two-bit state:

$$00 = \text{Void}, \quad 01 = \text{True}, \quad 10 = \text{False}, \quad 11 = \text{Both}.$$

Each register also carries a non-negative integer *paradox count* $\pi \in \mathbb{N}$, incremented by every engagement event.

Definition 2 (Belnap FOUR). The Belnap four-valued lattice $\mathbf{B}_4 = \{N, T, F, B\}$ is equipped with two partial orders. The *truth order* has $N, T, F < B$ and $N < T, F$. The *information order* has $N < T$ and $N < F$, with T, F incomparable and both below B . The *Belnap join* $\vee$ is the join in the information order:

$$N \vee x = x, \quad T \vee F = B, \quad B \vee x = B, \quad x \vee x = x.$$

In particular $B \vee B = B$ (idempotency of paradox).

The *Paraconsistent Universal Engine* (ParaEngine) is the TriPhaseRegister machine augmented with a belief map $\beta : \text{Reg} \rightarrow \mathbf{B}_4$ kept in sync with flux: $\beta(r) = N, T, F, B$ iff $r.\text{flux} = 00, 01, 10, 11$ respectively. The relevant IMASM opcodes are:

- ENGAGR $\%r_i$: sets $r_i.\text{flux} \leftarrow 11$, $\beta(r_i) \leftarrow B$, $r_i.\pi += 1$.
- FSPLIT $\%r_i \ \%r_j \ \%r_k$: applies ENGAGR to each of r_i, r_j, r_k in turn. Total: three engagement events, π incremented for each.

- FFUSE %r_i %r_j %r_k: sets $\beta(r_k) \leftarrow \beta(r_i) \vee \beta(r_j)$ (Belnap join). No engagement events; π unchanged.

FSPLIT is the Frobenius co-multiplication δ ; FFUSE is the multiplication μ .

2 The Kernel

The *Frobenius loop kernel* $\mathcal{K}$ is the following three-instruction program:

Listing 1: Frobenius loop kernel

```

1 K[0]  ENGAGR %r0          ; seed: force B on the root register
2 K[1]  FSPLIT %r0 %r1 %r2  ; delta: co-multiply -- two B-children
3 K[2]  FFUSE  %r1 %r2 %r0  ; mu: multiply -- fold back into root

```

The program counter is circular: after executing K[2] the engine sets $pc \leftarrow 0$ and continues. This is not an imposed wrapper; the `step()` method of the Universal Engine resets to zero at program end by design (*bootstrap loop closure*).

3 The Fixed Point Theorem

Theorem 1 (Permanence of B-state). *Let the engine start in any state. After the first complete cycle of $\mathcal{K}$, we have $\beta(r_0) = \beta(r_1) = \beta(r_2) = B$. This holds for every subsequent cycle.*

Proof. Track beliefs through one cycle, starting from an arbitrary initial state $\beta(r_0) = s_0 \in \mathbf{B}_4$.

1. ENGAGR %r0: $\beta(r_0) \leftarrow B$ (forced, regardless of s_0).
2. FSPLIT %r0 %r1 %r2: $\beta(r_0) \leftarrow B, \beta(r_1) \leftarrow B, \beta(r_2) \leftarrow B$ (each register is engaged independently).
3. FFUSE %r1 %r2 %r0: $\beta(r_0) \leftarrow \beta(r_1) \vee \beta(r_2) = B \vee B = B$ (Belnap join idempotency).

After cycle 1: $\beta(r_0) = \beta(r_1) = \beta(r_2) = B$. For any later cycle, ENGAGR at K[0] forces B on r_0 (already B), FSPLIT forces B on all three (already B), and FFUSE computes $B \vee B = B$. By induction the belief triple (B, B, B) is a fixed point of $\mathcal{K}$. $\square$

Remark 1. The Frobenius identity $\mu \circ \delta = \text{id}$ underlies step (3): FFUSE undoes the split performed by FSPLIT, mapping the B-pair back to the original B. Without idempotency of $\vee$ —i.e., in a lattice where $B \vee B \neq B$ —this fixed point need not exist. The Belnap lattice is precisely the setting in which paraconsistent contradiction is *self-sustaining* rather than explosive.

4 The Paradox Growth Theorem

Let $P(n)$ denote the total paradox count $\sum_{r \in \{r_0, r_1, r_2\}} r \cdot \pi$ after n complete cycles, starting from $P(0) = 0$.

Theorem 2 (Linear paradox growth). $P(n) = 4n$ for all $n \geq 0$.

Proof. Count engagement events per cycle. ENGAGR %r0 contributes 1 event ($r_0 \cdot \pi \leftarrow 1$). FSPLIT %r0 %r1 %r2 contributes 3 events (one per register). FFUSE contributes 0 events. The

per-cycle increment is therefore $1 + 3 + 0 = 4$. Since cycles are identical, $P(n) = 4n$ by induction. $\square$

Corollary 3. *The three registers accumulate paradox counts in the ratio $r_0.\pi : r_1.\pi : r_2.\pi = 2 : 1 : 1$ (both ENGAGR and FSPLIT fire on r_0 ; only FSPLIT fires on r_1 and r_2).*

5 Empirical Verification

The program `para_loop.py` runs $\mathcal{K}$ on a ParaEngine instance, reporting periodic snapshots. Two checkpoints were recorded during a single continuous session:

Checkpoint	Steps	Cycles n	Paradox P	$P - 4n$
t_1	625,700,000	208,566,666	834,266,668	+4
t_2	18,956,065,000	6,318,688,333	25,274,753,333	+1

The residuals $P - 4n \in \{0, 1, 2, 3, 4\}$ arise because snapshots are taken mid-cycle. At t_1 , residual +4 indicates capture immediately after FSPLIT (all four events of cycle $n + 1$ already logged); at t_2 , residual +1 indicates capture after ENGAGR alone. The residual is therefore a cycle-phase indicator, not a deviation from theorem 2.

In both snapshots: active registers = 3, fixed registers = 0, Belnap distribution = $(N=0, T=0, F=0, B=3)$. The machine ran from the first checkpoint to the second without interruption, a wall-clock duration of approximately 13.5 hours.

6 Remarks

Non-triviality. The result is not the observation that ENGAGR forces B and B persists; that would follow from any constant-forcing opcode. The non-trivial point is that FFUSE—whose semantic is the Belnap join, a *non-trivial* binary operation on $\mathbf{B}_4$ —preserves B specifically because B is the top element of the information order and $\vee$ is idempotent there. A different join (e.g. one that collapses $B \vee B$ to T) would break the fixed point. The Belnap lattice is the minimal structure in which paradox is both non-explosive and self-sustaining.

Paraconsistency vs. classical contradiction. In a classical machine, a register holding both True and False would represent an error state and computation would halt or diverge. Here contradiction is stabilized in place: the engine continues executing, paradox count grows as a clean linear function, and no classical truth value is ever witnessed. The machine does not *resolve* the contradiction; it *runs on it*.

7 Resolution: IFIX Cannot Collapse the Loop

We answer the open question stated above. There are two independent reasons IFIX cannot drive the system to all- T .

Theorem 4 (IFIX absorption by B). *Let $\mathcal{K}'$ be any circular kernel that retains ENGAGR and FSPLIT over the registers feeding into FFUSE. Then no finite sequence of IFIX instructions in $\mathcal{K}'$ can produce a state in which all registers hold belief T at the end of any cycle.*

Proof. Two cases cover all placements of IFIX in $\mathcal{K}'$.

Case A: IFIX fires on a register r before FSPLIT touches r in the same cycle. FSPLIT calls `engage()` on every argument register. `engage()` sets `flux` $\leftarrow$ 11 without consulting the `is_fixed` property; it also updates belief to B via `_set_belief`. Therefore the IFIX- T state is overridden by the subsequent FSPLIT, and r returns to B before FFUSE fires.

Case B: IFIX fires on a register r after FSPLIT and before FFUSE, so r enters FFUSE with belief T . At least one other FFUSE input is a register touched by FSPLIT this cycle (by assumption that FSPLIT feeds FFUSE); that register holds B . By the Belnap join: $T \vee B = B$. Therefore FFUSE outputs B , not T .

In both cases the FFUSE output that feeds the root r_0 is B . ENGAGR at the start of the next cycle forces r_0 back to B regardless. By induction over cycles, the system never reaches all- T . $\square$

Remark 2 (Register-level paraconsistency in K3). A third phenomenon appears when IFIX fires on r_0 before ENGAGR in the same cycle (**K3** below). IFIX sets `r.value` = FIXED and belief $\leftarrow T$. ENGAGR then sets `flux` = 11 and belief $\leftarrow B$, but does *not* clear `r.value`. The register therefore simultaneously satisfies the linear type constraint (`value` = FIXED, installed by IFIX) and the paradox belief state (`flux` = 11, installed by ENGAGR). This is paraconsistency at the register level: the FIXED constraint and the B-state coexist without contradiction. The machine does not resolve the tension; it carries both indefinitely.

Experimental confirmation

Three extended kernels were executed for 5×10^5 cycles each (script `ifix_experiment.py`):

Kernel	IFIX placement	Belnap at cycle end	r_0 belief
K1	on r_1 , after FSPLIT	$N=0$ $T=1$ $F=0$ $B=2$	B
K2	on r_2 , after FSPLIT	$N=0$ $T=1$ $F=0$ $B=2$	B
K3	on r_0 , before ENGAGR	$N=0$ $T=0$ $F=0$ $B=3$	B [FIXED]

In K1 and K2 the IFIXed register ends each cycle at T (IFIX fires last on it), but r_0 remains B in all cases via Case B of theorem 4. In K3 the FIXED marker persists on r_0 alongside `flux` = 11 and belief = B , confirming the register-level paraconsistency described in the Remark above.

Corollary 5. *The Frobenius loop is IFIX-stable: no circular extension of $\mathcal{K}$ that preserves ENGAGR and FSPLIT can collapse the system to all- T . The paraconsistent fixed point is robust to the linear type constraint.*